

# Spring Boot – Services et logique métier

Les **Services** permettent de centraliser la logique métier et d'assurer un **couplage faible** entre les controllers et l'accès aux données.

# Pourquoi utiliser des Services

Un controller doit :

- Recevoir la requête HTTP
- Appeler la logique métier
- Retourner une réponse

Un controller **ne doit pas** :

- Contenir de logique métier
- Accéder directement aux repositories

## Objectif :

- Séparer les responsabilités
- Faciliter l'évolution et les tests
- Éviter le couplage fort

## Mauvaise pratique (à éviter)

```
@RestController
public class JoueurController {

    private final JoueurRepository joueurRepository;

    public JoueurController(JoueurRepository joueurRepository) {
        this.joueurRepository = joueurRepository;
    }

    @GetMapping("/joueurs")
    public List<Joueur> getJoueurs() {
        return joueurRepository.findAll();
    }
}
```

## Problèmes :

- Logique métier absente
- Controller dépend directement de la base
- Code difficile à faire évoluer

# Introduction du Service (bonne pratique)

Le Service agit comme un pont entre :

- le controller
- le repository

```
@Service
public class JoueurService {

    private final JoueurRepository joueurRepository;

    public JoueurService(JoueurRepository joueurRepository) {
        this.joueurRepository = joueurRepository;
    }

    public List<Joueur> recupererTousLesJoueurs() {
        return joueurRepository.findAll();
    }
}
```

Le service :

- Contient la logique métier
- Appelle les repositories
- Ne dépend pas du HTTP

## Controller appelant le Service

```
@RestController
public class JoueurController {

    private final JoueurService joueurService;

    public JoueurController(JoueurService joueurService) {
        this.joueurService = joueurService;
    }

    @GetMapping("/joueurs")
    public ApiResponse<List<Joueur>> getJoueurs() {
        return joueurService.getTousLesJoueurs();
    }
}
```

Le controller :

- Ne connaît pas la base de données
- Se concentre sur l'API REST

# Standardiser les réponses métier

Toutes les réponses JSON doivent contenir :

- un **code métier**
- un **message métier**
- une **donnée générique**

Objectif :

- Réponses cohérentes
- Facilité de consommation côté client

## Classe de réponse générique

```
public class ApiResponse<T> {  
  
    public String code;  
    public String message;  
    public T data;  
  
    public ApiResponse(String code, String message, T data) {  
        this.code = code;  
        this.message = message;  
        this.data = data;  
    }  
}
```

# Logique métier dans le Service

```
@Service
public class JoueurService {

    private final JoueurRepository joueurRepository;

    public JoueurService(JoueurRepository joueurRepository) {
        this.joueurRepository = joueurRepository;
    }

    public ApiResponse<List<Joueur>> getTousLesJoueurs() {
        List<Joueur> joueurs = joueurRepository.findAll();

        if (joueurs.isEmpty()) {
            return new ApiResponse<>("701", "Aucun joueur trouvé", joueurs);
        }

        return new ApiResponse<>("200", "Liste des joueurs récupérée avec succès", joueurs);
    }
}
```

La logique métier :

- Décide du code et du message
- Prépare la réponse finale
- Ne dépend pas du controller

## Exemple de réponse JSON

```
{
  "code": "JOUER_000",
  "message": "Liste des joueurs récupérée avec succès",
  "data": [
    {
      "id": 1,
      "nom": "Alice",
      "score": 150
    }
  ]
}
```

# Conclusion

- Le controller ne doit jamais appeler un repository
- Le service centralise la logique métier
- Le couplage faible améliore la maintenabilité
- Les réponses API sont standardisées et génériques